

Tablice dynamiczne – wprowadzenie

W jaki sposób zadeklarować tablicę o rozmiarze, który dopiero np. wczytamy z klawiatury?

Tak nie wolno robić!!!!

```
int rozmiar;  
cin >> rozmiar;  
int tab[ rozmiar ];
```

Rozmiar tablicy statycznej możemy zadeklarować tylko wtedy, gdy w momencie kompilacji znamy wartość dla tego rozmiaru. W przykładzie powyżej rozmiar poznajemy dopiero w trakcie działania programu. Niedozwolona jest zatem deklaracja tablicy jak powyżej!!! Należy użyć **tablicy dynamicznej**. (kolejny rozdział)

//-----

Bardzo duuuuuuuuuuuuża tablica.

```
int tab[ 1000000 ];
```

Deklaracja tablicy jak powyżej wygeneruje błąd programu. Suma rozmiarów (w bajtach) dla wszystkich zmiennych (i tablic) użytych w programie musi się zmieścić w specjalnym obszarze pamięci zwanym STOSem. Nie jest to duża wartość, zazwyczaj max kilka MB. Na szczęście pamięci RAM mamy pod dostatkiem, więc możemy z niej korzystać ile tylko chcemy.

Aby zadeklarować tablicę o bardzo dużej liczbie elementów należy użyć **tablicy dynamicznej**. (kolejny rozdział)

W uproszczeniu chodzi o to, aby całą tablicę zadeklarować gdzieś w pamięci RAM (w jaki sposób, to nas nie interesuje, bo tym zajmie się system operacyjny), a w pamięci programu (na STOSie) zostawić jedynie wskaźnik – skrót – lokalizację do pamięci RAM, gdzie ta tablica się znajduje. Wskaźniki poznaliśmy przy okazji funkcji z parametrami.

Główna wada tablic dynamicznych.

Czas dostępu do globalnego RAMu jest nieco dłuższy, niż do pamięci cache procesora (zazwyczaj da się wkopiować cały STOS programu do niej), czy do jego rejestrów. Przez co programy wymagające dużej liczby operacji na dużej liczbie danych wykonują się wolniej. Jednak biorąc pod uwagę sumę zalet wynikającej z dynamicznej alokacji pamięci, to ta „drobna” wada może zostać

pominięta.

Tablice dynamiczne

Przykładowa deklaracja tablicy dynamicznej (poprzez wczytanie jej rozmiaru z klawiatury):

```
int n;  
cin >> n;  
int *Tab = new int[ n ];
```

Za skomplikowane? To rozbijmy to na 2 linie:

```
int *Tab;  
Tab = new int[ n ];
```

Co oznacza powyższy zapis?

Zadeklaruj zmienną typu **wskaźnik na element** (lub elementy) typu **int**. **Znajdź i zarezerwuj w pamięci RAM (instrukcja „new”) jakiś wolny obszar pamięci**, który będzie w stanie pomieścić **tablicę (operatory [])** o **n elementach** ([n]), każdy z nich typu **int**.

//-----

Teraz możemy w programie używać tej tablicy tak samo, jak tablic statycznych.

//-----

Aby nie było wycieków pamięci należy zawsze pamiętać o zwolnieniu pamięci, gdy dany obiekt nie będzie nam już potrzebny.

```
delete [] Tab;  
Tab = NULL;
```

Wykasuj z pamięci RAM (instrukcja „delete”) całkowity blok pamięci, czyli wszystkie (znaczniki []) komórki tablicy o nazwie Tab.

Ze względów bezpieczeństwa dobrą praktyką programistyczną jest ustawienie wskaźnika na **NULL** (na **zero**) tuż po wykasowaniu elementu z pamięci. Przyjęło się, że coś, co wskazuje na zerową komórkę pamięci to nie istnieje.

//-----